1. System Architecture

1.1. Classes & Responsibilities

Database: Manages the API for table and tuple operations. Responsible for the catalog, table mappings, and lifetime of the storage engine.

FileManager: Handles I/O operations.

BufferPool: Manage a fixed number of buffer frames, page caching, page pinning, dirty-page tracking, and LRU eviction

Page: Represents the 4096 byte data page. Allows the referenced retrieval of the page bytes, the page ID, the page number, the tableID, and the size. Also allows a copy retrieval of the page bytes.

DataPage: Logical representation of a Page containing tuple data. Contains a reference to the page bytes and is responsible for logical writing before physical writing.

TableHeader: Logical representation of a table's metadata including the table name, the table ID, and schema information.

Tuple: Represents a database record allowing the retrieval of tuple values.

Schema: Represents the ordering, names, and types of a table.

RID: Identifies a tuple using its table ID, page number, and tuple slot.

2. Storage Design

2.1. File & Page Layout

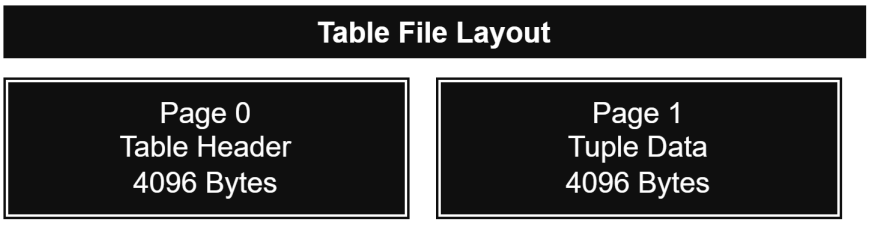
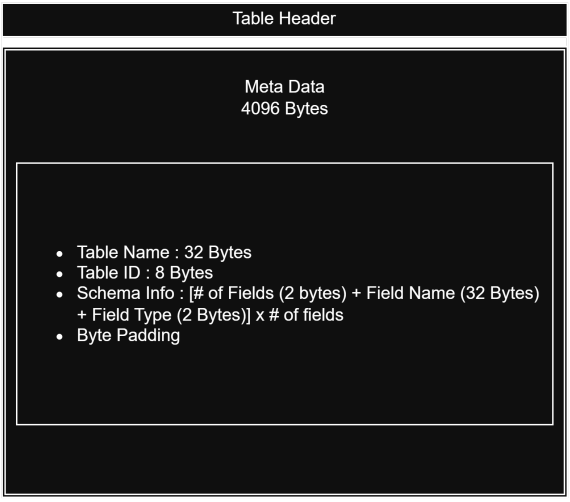


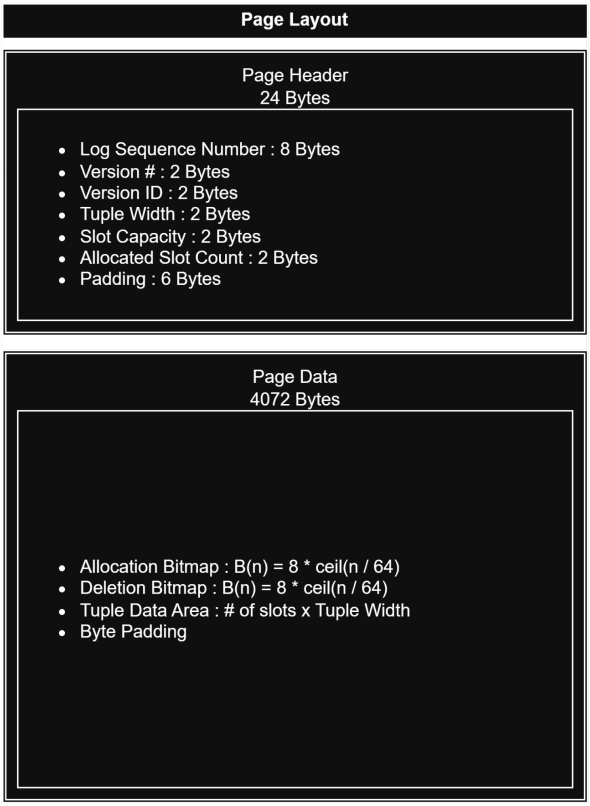
Table Header -> Page 0



Schema Info Calculation Info:

A schema is made up of columns with a given name and type. 2 Bytes are going to indicate to the decoder how many columns make up the schema. 32 Bytes are stored for the name string since the engine supports STRING32. FieldType corresponds to a 2 byte enum that indicates how many bytes should be read to fill the field. Then this process will be repeated for as many columns there are in the schema. Finally, padding will be added to reach the 4096 bytes.

Page -> Page 1…n (Tuple Data)



2.2. Tuple Capacity Calculation

A page has a total fixed-capacity of 4096 bytes due to the project's constraints. The slot capacity calculation is given as such:

$H + 2 \times B(n) + n \times R <= 4096$

Where $B(n) = 8 \times \text{ceil}(n / 64)$

H = header bytes

n = slot capacity

R = tuple width

I have chosen the header bytes size to be 24 bytes. So then to find the slot capacity is a question of finding n such that it does not go over the 4096 bytes limit. Code-wise this is done in a brute force manner.

Max Tuple Width Calculation

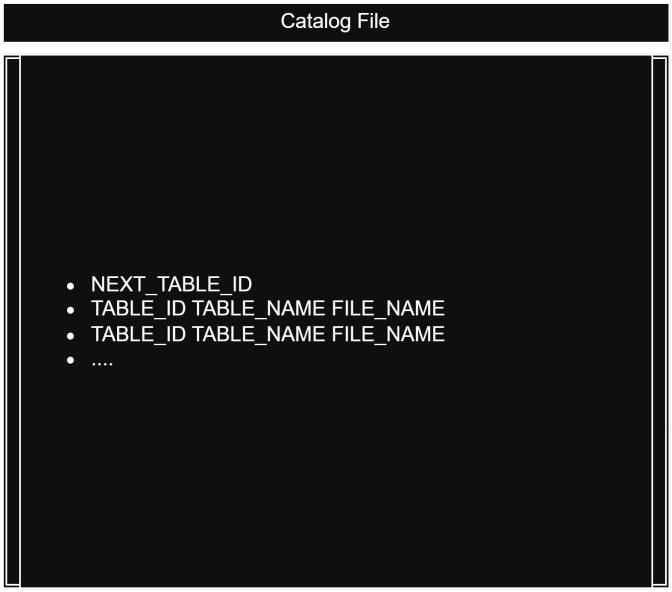
The maximum tuple width is going to have a slot capacity = 1.

Plugging n = 1 into the equation nets R <= 4056. Therefore, the maximum tuple width is 4056.

Max Tuple Slot Capacity Calculation

Code-wise, the maximum tuple slot capacity calculation is done using a for loop while plugging in increasing n-values into the previous equation. H & R are known which allows this brute-force method.

2.3. Catalog File



This is a metadata file that is loaded upon initialization to fill in lookup maps to look up

TableName → TableID

TableID → Table_File_Name

3. Buffer Pool Design

3.1. Frame

Upon creation, a buffer pool is given a static amount of frames that it could allocate to pages. A buffer frame contains:

- isOccupied flag
- Page (4096 Bytes that it owns)
- pinCount
- dirtyFlag

When a page is loaded into a frame, its bytes are copied into it and a logical Page object is decoded from these bytes and held inside the frame.

3.2. Pinning

A page is pinned when the fetchPage function is called.

A page is unpinned when the unPinPage function is called. That function is also supplied with a boolean to indicate if any changes had been made (sets the dirty flag when true).

3.3. LRU

An LRU list is implemented using std::list, with the back of the list corresponding to the Most Recently Used. The LRU list is managed via the following rules:

- Page is fetched & not in pool & buffer frame available

  → The unused buffer frame is allocated & its index is added to the back of the LRU List.

- Page is fetched & is in pool

  → The frame index the page is allocated to is removed then replaced to the back of the LRU List.

- Page is fetched & not in pool & buffer frame is not available

  → The most front-most unpinned frame in the LRU list is evicted. The frame is allocated to the page. The frame is added to the back of the LRU List.

4. API & Ownership

4.1. API

The storage engine is accessed through the Database class, defined in database.hpp. The API supports functionality for table creation, tuple insertion, tuple retrieval, tuple modification, tuple deletion, table scanning, and persistent storage management.

Database Initialization

Database(const std::filesystem::path& directory, std::size_t bufferFrameCount = 3);

Initializes the storage engine using the specified storage directory and allocates a fixed supplied number of buffer pool frames. During initialization, the database loads its catalog file to construct in-memory mappings between table names, IDs, and their filenames.

Table Management

void createTable(const std::string& tableName, const Schema& schema);

Creates a dedicated file for a new table and stores its schema & other metadata information in the table header. Also adds it to the mapping.

TableHeader queryTableInfo(std::string& tableName) const;

Grabs a copy of the table's metadata from its table header.

Tuple Management

RID insertTuple(const std::string& tableName, Tuple tuple);

Encodes a tuple, locates an available slot, writes tuple to that slot, returns the RID identifying the inserted record.
Returns an RID with the tuple slot -1 when full or error occurs.

Tuple readTuple(const std::string& tableName, RID recordID);

Uses the provided RID & table to look up a tuple within its page. Decodes its retrieved bytes and returns a copy of the Tuple.
Returns an empty tuple when a read error occurs

RID updateTuple(const std::string& tableName,RID recordID, Tuple updatedTuple);

Uses the provided RID & table to look up a tuple within its page. Overwrites its stored bytes with the encoded updatedTuple. The original RID is returned.
Returns an RID with the tuple slot -1 when full or error occurs.

void deleteTuple(const std::string& tableName, RID recordID);

Performs a logical deletion by setting the tuple's corresponding deletion bit within the page.

std::vector<std::pair<Tuple, RID>> scan(const std::string& tableName);

Iterates through the table's stored tuples, decodes them, and returns a vector containing each tuple paired with its RID

Buffer Pool & Persistence

void flushAllPage();

Writes the dirty pages from the buffer pool back to their corresponding files.

Database Destruction

~Database();

Invokes flushAllPages() to attempt to write back any dirty pages in the pool.

4.2. Object Ownership

Database: Owns FileManager, BufferPool, Lookup Maps, Data Directory Path

Responsible for the lifetime of the storage engine. When retrieving Tuples, it only receives copies and never has direct access to the page bytes.

BufferPool: Owns BufferFrames, Page data

5. Failure Handling & Correctness Justifications

5.1. Pinned Frame Protection

The buffer pool uses pins to ensure that a page currently being accessed cannot be evicted. fetchPage() increments the pin and is only decremented when upinPage() is called.

5.2. Dirty Page Writeback Failure Handling

Using a dirty flag, the buffer pool tracks which pages have been modified. Dirty pages are written back under two scenarios:

1. Frame Eviction: When a dirty frame is evicted, the contents are written back before frame reuse.

2. Explicit Flushing: When flushAllPages() is called , the buffer pool attempts to write back all dirty pages.

A dirty page is only marked as clean if its write operation succeeds. So if the write operation fails, the dirty frame is not reused.

6. Limitations

1. The storage engine does not support multiple data pages.
2. The storage engine will not reuse slots that have been deleted
3. Variable-length fields are not supported
4. NULL values are not supported
5. No concurrency control implementations